

Лабораторная работа №9

Бинарное дерево. Обходы. BST

Цель работы:

1. Реализовать структуру данных «бинарное дерево» и изучить различные способы его обхода (прямой, симметричный, обратный, уровневый)
2. Реализовать бинарное дерево поиска (BST) с операциями вставки, удаления и поиска
3. Освоить применение рекурсии для работы с деревьями
4. Проанализировать сложность операций в BST
5. Научиться визуализировать структуру дерева

Теоретическое обоснование

1. Бинарное дерево

Бинарное дерево - это иерархическая структура данных, в которой каждый узел имеет не более двух потомков: левый и правый.

Терминология:

Термин	Определение
Корень (root)	Верхний узел дерева
Лист (leaf)	Узел, не имеющий потомков
Внутренний узел	Узел, имеющий хотя бы одного потомка
Высота дерева	Максимальное расстояние от корня до листа
Уровень (level)	Расстояние от корня (корень - уровень 0)

Визуализация бинарного дерева:

```
      10 (корень)
     /  \
    5    15
   /\   \
  3 7   20
   /\   /
  6 18
```

2. Обходы бинарного дерева

Вид обхода	Порядок посещения	Результат для дерева выше
Прямой (pre-order)	Корень → Левый → Правый	10, 5, 3, 7, 6, 15, 20, 18
Симметричный (in-order)	Левый → Корень → Правый	3, 5, 6, 7, 10, 15, 18, 20
Обратный (post-order)	Левый → Правый → Корень	3, 6, 7, 5, 18, 20, 15, 10
Уровневый (level-order)	По уровням слева направо	10, 5, 15, 3, 7, 20, 6, 18

// Прямой обход (pre-order)

```
void preOrder(Node* node) {  
    if (node == nullptr) return;  
    cout << node->data << " ";  
    preOrder(node->left);  
    preOrder(node->right);  
}
```

// Симметричный обход (in-order)

```
void inOrder(Node* node) {  
    if (node == nullptr) return;  
    inOrder(node->left);  
    cout << node->data << " ";  
    inOrder(node->right);  
}
```

// Обратный обход (post-order)

```
void postOrder(Node* node) {  
    if (node == nullptr) return;  
    postOrder(node->left);  
    postOrder(node->right);  
    cout << node->data << " ";  
}
```

// Уровневый обход (level-order) с использованием очереди

```

void levelOrder(Node* root) {
    if (root == nullptr) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        cout << current->data << " ";
        if (current->left) q.push(current->left);
        if (current->right) q.push(current->right);
    }
}

```

3. Бинарное дерево поиска (BST)

Бинарное дерево поиска (BST) - это бинарное дерево, в котором для каждого узла выполняется условие:

Все ключи в левом поддереве < ключ узла < все ключи в правом поддереве

Свойства BST:

Симметричный обход даёт отсортированную последовательность

Поиск, вставка и удаление в среднем выполняются за $O(\log n)$

Визуализация BST:

```

      50
     /  \
    30   70
   /\  /\
  20 40 60 80

```

4. Операции с BST

Поиск $O(\log n)$ в среднем

```

Node* search(Node* root, int key) {
    if (root == nullptr || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

```

Вставка $O(\log n)$ в среднем

cpp

```

Node* insert(Node* root, int key) {
    if (root == nullptr) return new Node(key);
    if (key < root->data) root->left = insert(root->left, key);
    else if (key > root->data) root->right = insert(root->right, key);
}

```

```
return root;
```

```
}
```

Удаление $O(\log n)$ в среднем

Удаление узла из BST имеет три случая:

Случай	Описание	Действие
1	Удаляемый узел - лист	Просто удаляем
2	Узел имеет одного потомка	Заменяем удаляемый узел его потомком
3	Узел имеет двух потомков	Находим минимальный элемент в правом поддереве (или максимальный в левом), заменяем значение, удаляем тот узел

```
Node* findMin(Node* root) {  
    while (root->left != nullptr) root = root->left;  
    return root;  
}
```

```
Node* remove(Node* root, int key) {  
    if (root == nullptr) return nullptr;  
  
    if (key < root->data) root->left = remove(root->left, key);  
    else if (key > root->data) root->right = remove(root->right, key);  
    else {  
        // Случай 1: лист  
        if (root->left == nullptr && root->right == nullptr) {  
            delete root;  
            return nullptr;  
        }  
        // Случай 2: один потомок  
        else if (root->left == nullptr) {  
            Node* temp = root->right;  
            delete root;  
            return temp;  
        }  
        else if (root->right == nullptr) {
```

```

        Node* temp = root->left
        delete root;
        return temp;
    }
    // Случай 3: два потомка
    else {
        Node* minNode = findMin(root->right);
        root->data = minNode->data;
        root->right = remove(root->right, minNode->data);
    }
}
return root;
}

```

5. Вычисление характеристик дерева

```

// Высота дерева
int height(Node* root) {
    if (root == nullptr) return 0;
    return 1 + max(height(root->left), height(root->right));
}

// Количество узлов
int countNodes(Node* root) {
    if (root == nullptr) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

// Количество листьев
int countLeaves(Node* root) {
    if (root == nullptr) return 0;
    if (root->left == nullptr && root->right == nullptr) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}

```

Примеры выполнения практической части

Пример 1. Бинарное дерево и обходы (базовый уровень)

Задача: Создать бинарное дерево и выполнить все четыре обхода.

```

#include <iostream>
#include <queue>

```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
    Node(int val) : data(val), left(nullptr), right(nullptr) {}  
};
```

```
class BinaryTree {  
private:  
    Node* root;
```

```
void preOrder(Node* node) {  
    if (node == nullptr) return;  
    cout << node->data << " ";  
    preOrder(node->left);  
    preOrder(node->right);  
}
```

```
void inOrder(Node* node) {  
    if (node == nullptr) return;  
    inOrder(node->left);  
    cout << node->data << " ";  
    inOrder(node->right);  
}
```

```
void postOrder(Node* node) {  
    if (node == nullptr) return;  
    postOrder(node->left);  
    postOrder(node->right);  
    cout << node->data << " ";  
}
```

```
void destroy(Node* node) {  
    if (node == nullptr) return;  
    destroy(node->left);  
    destroy(node->right);  
    delete node;  
}
```

```
public:
```

```
BinaryTree() : root(nullptr) {}
```

```
~BinaryTree() { destroy(root); }
```

```
void buildSample() {  
    root = new Node(10);  
    root->left = new Node(5);  
    root->right = new Node(15);  
    root->left->left = new Node(3);  
    root->left->right = new Node(7);  
    root->right->right = new Node(20);  
    root->left->right->left = new Node(6);  
    root->right->right->left = new Node(18);  
}
```

```
void preOrder() { preOrder(root); cout << endl; }
```

```
void inOrder() { inOrder(root); cout << endl; }
```

```
void postOrder() { postOrder(root); cout << endl; }
```

```
void levelOrder() {  
    if (root == nullptr) return;  
    queue<Node*> q;  
    q.push(root);  
    while (!q.empty()) {  
        Node* current = q.front();  
        q.pop();  
        cout << current->data << " ";  
        if (current->left) q.push(current->left);  
        if (current->right) q.push(current->right);  
    }  
    cout << endl;  
}
```

```
int main() {  
    BinaryTree tree;  
    tree.buildSample();  
  
    cout << "Прямой обход (pre-order): ";  
    tree.preOrder();  
    cout << "Симметричный (in-order): ";  
    tree.inOrder();  
}
```

```

cout << "Обратный (post-order):  ";
tree.postOrder();
cout << "Уровневый (level-order):  ";
tree.levelOrder();

return 0;
}

```

Ожидаемый вывод:

Прямой обход (pre-order): 10 5 3 7 6 15 20 18

Симметричный (in-order): 3 5 6 7 10 15 18 20

Обратный (post-order): 3 6 7 5 18 20 15 10

Уровневый (level-order): 10 5 15 3 7 20 6 18

Пример 2. Бинарное дерево поиска (средний уровень)

Задача: Реализовать BST с операциями вставки, поиска, удаления и вывода.

```

#include <iostream>
#include <queue>

using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class BST {
private:
    Node* root;

    Node* insert(Node* node, int key) {
        if (node == nullptr) return new Node(key);
        if (key < node->data) node->left = insert(node->left, key);
        else if (key > node->data) node->right = insert(node->right, key);
        return node;
    }

    Node* search(Node* node, int key) {

```

```

    if (node == nullptr || node->data == key) return node;
    if (key < node->data) return search(node->left, key);
    return search(node->right, key);
}

```

```

Node* findMin(Node* node) {
    while (node->left != nullptr) node = node->left;
    return node;
}

```

```

Node* remove(Node* node, int key) {
    if (node == nullptr) return nullptr;

    if (key < node->data) node->left = remove(node->left, key);
    else if (key > node->data) node->right = remove(node->right, key);
    else {
        if (node->left == nullptr && node->right == nullptr) {
            delete node;
            return nullptr;
        }
        else if (node->left == nullptr) {
            Node* temp = node->right;
            delete node;
            return temp;
        }
        else if (node->right == nullptr) {
            Node* temp = node->left;
            delete node;
            return temp;
        }
        else {
            Node* minNode = findMin(node->right);
            node->data = minNode->data;
            node->right = remove(node->right, minNode->data);
        }
    }
    return node;
}

```

```

void inOrder(Node* node) {
    if (node == nullptr) return;
    inOrder(node->left);
}

```

```

        cout << node->data << " ";
        inOrder(node->right);
    }

    void destroy(Node* node) {
        if (node == nullptr) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }

    int height(Node* node) {
        if (node == nullptr) return 0;
        return 1 + max(height(node->left), height(node->right));
    }

public:
    BST() : root(nullptr) {}

    ~BST() { destroy(root); }

    void insert(int key) { root = insert(root, key); }

    bool search(int key) { return search(root, key) != nullptr; }

    void remove(int key) { root = remove(root, key); }

    void print() {
        inOrder(root);
        cout << endl;
    }

    int getHeight() { return height(root); }
};

int main() {
    BST tree;

    tree.insert(50);
    tree.insert(30);
    tree.insert(70);
    tree.insert(20);

```

```

tree.insert(40);
tree.insert(60);
tree.insert(80);

cout << "Симметричный обход BST: ";
tree.print();
cout << "Высота дерева: " << tree.getHeight() << endl;

cout << "Поиск 40: " << (tree.search(40) ? "найден" : "не найден") <<
endl
cout << "Поиск 100: " << (tree.search(100) ? "найден" : "не найден") <<
endl

tree.remove(30);
cout << "После удаления 30: ";
tree.print();

return 0;
}

```

Ожидаемый вывод:

Симметричный обход BST: 20 30 40 50 60 70 80

Высота дерева: 3

Поиск 40: найден

Поиск 100: не найден

После удаления 30: 20 40 50 60 70 80

Индивидуальные задания

Часть 1. Бинарное дерево и обходы

Каждый вариант содержит задание реализовать бинарное дерево с указанными операциями:

№ вар.	Обязательные операции	Дополнительное задание
1	preOrder, inOrder, postOrder	Вычислить количество узлов
2	preOrder, inOrder, postOrder	Вычислить количество листьев
3	preOrder, inOrder, postOrder	Вычислить высоту дерева

№ вар.	Обязательные операции	Дополнительное задание
4	preOrder, inOrder, postOrder	Найти максимальный элемент
5	preOrder, inOrder, postOrder	Найти минимальный элемент
6	preOrder, inOrder, postOrder, levelOrder	Найти сумму всех элементов
7	preOrder, inOrder, postOrder, levelOrder	Найти среднее арифметическое
8	preOrder, inOrder, postOrder, levelOrder	Проверить, является ли дерево вырожденным
9	preOrder, inOrder, postOrder, levelOrder	Проверить, является ли дерево полным
10	preOrder, inOrder, postOrder, levelOrder	Проверить, является ли дерево совершенным
11	preOrder, inOrder, postOrder, levelOrder	Построить зеркальное отображение (swap левого и правого)
12	preOrder, inOrder, postOrder, levelOrder	Найти путь от корня до заданного узла
13	preOrder, inOrder, postOrder, levelOrder	Найти наибольшую сумму на уровне
14	preOrder, inOrder, postOrder, levelOrder	Найти диаметр дерева (максимальное расстояние между узлами)
15	preOrder, inOrder, postOrder, levelOrder	Проверить, является ли дерево сбалансированным (разница высот ≤ 1)

Часть 2. Бинарное дерево поиска (BST)

Каждый вариант содержит задание реализовать BST с указанными операциями:

№ вар.	Обязательные операции	Дополнительное задание
1	insert, search, inOrder	Удаление (все случаи)
2	insert, search, inOrder	Найти k-й наименьший элемент
3	insert, search, inOrder	Найти k-й наибольший элемент
4	insert, search, inOrder	Найти наименьший общий предок (LCA)
5	insert, search, inOrder	Найти преемника (следующий по порядку)
6	insert, search, inOrder	Найти предшественника (предыдущий по порядку)
7	insert, search, inOrder, remove	Визуализация дерева в консоли
8	insert, search, inOrder, remove	Проверка, является ли дерево BST
9	insert, search, inOrder, remove	Построение BST из отсортированного массива (сбалансированно)
10	insert, search, inOrder, remove	Построение BST из префиксного обхода
11	insert, search, inOrder, remove	Восстановление BST из двух переставленных узлов
12	insert, search, inOrder, remove	Подсчёт узлов в заданном диапазоне
13	insert, search, inOrder, remove	Вычислить сумму узлов в заданном диапазоне

№ вар.	Обязательные операции	Дополнительное задание
14	insert, search, inOrder, remove	Объединить два BST
15	insert, search, inOrder, remove	Проверить, является ли BST сбалансированным

Часть 3. Применение BST

Задание для всех вариантов:

Реализовать программу, которая:

1. Создаёт BST из последовательности случайных чисел ($N = 20$)
2. Выполняет симметричный обход и выводит отсортированную последовательность
3. Находит минимальный и максимальный элемент
4. Находит высоту дерева
5. Удаляет элемент, введённый пользователем, и выводит дерево после удаления

Контрольные вопросы

1. Что такое бинарное дерево? Из каких элементов оно состоит?
2. В чём отличие бинарного дерева от бинарного дерева поиска (BST)?
3. Какие существуют виды обхода бинарного дерева? В чём их особенности?
4. Какой обход даёт отсортированную последовательность для BST?
5. Какова сложность операций поиска, вставки и удаления в BST? От чего она зависит?
6. Что происходит с BST, если вставлять элементы в отсортированном порядке?
7. Какие три случая нужно рассмотреть при удалении узла из BST?
8. Как найти наименьший элемент в BST? Наибольший?
9. Как вычислить высоту бинарного дерева рекурсивно?
10. Что такое наименьший общий предок (LCA) двух узлов в дереве?
11. Как проверить, является ли бинарное дерево BST?
12. Что такое сбалансированное дерево? Почему это важно для производительности?
13. Как преобразовать несбалансированное BST в сбалансированное?
14. Как восстановить дерево по массиву обхода (in-order + pre-order)?

15.Как реализовать итеративный (не рекурсивный) обход бинарного дерева?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (бинарное дерево, обходы, BST)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода, визуализация дерева)

Анализ сложности операций

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Бинарное дерево и обходы	30
1.1	Структура узла и класс дерева	10
1.2	Реализация всех обходов	15
1.3	Вычисление характеристик дерева	5
2	Бинарное дерево поиска (BST)	40
2.1	Реализация вставки	10
2.2	Реализация поиска	10
2.3	Реализация удаления (все три случая)	15
2.4	Корректная работа с памятью (деструктор)	5
3	Применение и анализ	15
3.1	Демонстрационная программа	5

№	Критерий	Баллы
3.2	Анализ сложности операций	5
3.3	Выводы и сравнения	5
4	Оформление и отчёт	15
4.1	Код отформатирован, есть комментарии	5
4.2	Есть ответы на контрольные вопросы	5
4.3	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Некорректное удаление узла с двумя потомками	-15
Утечка памяти (не освобождены узлы)	-15
Некорректные обходы дерева	-10
Нет обработки пустого дерева	-5
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74

Оценка	Баллы
Неудовлетворительно	менее 60